Cresta Senior ML Engineer

Onsite interview question bank (partial)

01. Cresta Senior MLE Interview Loop: Format, Rounds, Culture and the Senior Bar

- Format notes (expected, unverified)
- Questions
- References

01. Cresta Senior MLE Interview Loop: Format, Rounds, Culture and the Senior Bar

> **Research status (read first).** This chapter was written in a run where live research was blocked. The shared web-search budget was used up, and the egress proxy refused connections to cresta.com, glassdoor.com, teamblind.com and jobs.ashbyhq.com. **No candidate reports were retrieved, so no question below is tagged [Reported].** Every question is tagged [Likely]: an inference from Cresta's public product line (real-time contact-center AI) and the usual shape of senior MLE loops at Series C/D AI startups. Before the onsite, check the format against your recruiter's prep email and against Glassdoor and Blind. Ask the recruiter directly for the round list, the duration of each round, and whether coding is done in an IDE or a shared doc.

Format notes (expected, unverified)

Background context from general knowledge, not fetched in this run: Cresta builds generative AI for contact centers. Its products cover real-time agent guidance (Agent Assist), post-call analytics (Conversation Intelligence and automated QA), knowledge assist, coaching, and AI/virtual agents. That means streaming ASR, real-time LLM inference, per-customer customization and enterprise data constraints all apply at the same time. Expect the loop to test whether you can ship ML that works under latency, cost and customer-specific accuracy requirements, not only whether you know the models.

Typical structure for this kind of role (confirm with the recruiter):

Stage	Likely duration	Interviewer	What the senior bar looks like
Recruiter screen	30 min	Recruiter	Motivation for Cresta and contact-center AI, level calibration, compensation, timeline
Technical screen	45-60 min	Engineer / MLE	Practical coding (data processing, Python, possibly an ML-flavored implementation); clean, tested, talks through tradeoffs
Hiring-manager call	30-45 min	ML/eng manager	Scope of past work, ownership, fit for a specific team (e.g., agent assist, AI agent, analytics, ML platform)
Onsite: coding	45-60 min	Engineer	Production-quality code, edge cases, complexity; at senior level, extensible design without prompting
Onsite: ML system design	60 min	Senior MLE / staff	End-to-end design of an LLM/ASR-powered contact-center feature: data, labeling, modeling, evaluation, serving latency, monitoring, rollout
Onsite: ML depth / project deep dive	45-60 min	Senior MLE / research	Deep ownership of one past project; justifies choices with metrics; knows the failure modes

Stage	Likely duration	Interviewer	What the senior bar looks like
Onsite: behavioral / values	45 min	Manager or cross-functional	Ownership, customer focus, ambiguity, mentorship, cross-team influence
Possible: take-home or practical ML exercise	variable	n/a	Some startups use a take-home or a live practical round (e.g., an evaluation or fine-tuning exercise); ask whether one applies

What usually separates senior from mid-level in this kind of loop:

- Drives the design conversation: sets the requirements and metrics before choosing a model.
- Reasons about the whole system, including latency budgets for real-time features, cost per conversation, multi-tenant customization, data privacy and PII, and online/offline evaluation gaps.
- Has stories with measurable business impact, and has influenced people beyond their own team (product, customers, other teams).
- Raises risks first: hallucination, drift, label noise, customer-specific regressions.

Questions

A. Recruiter screen and motivation

1. Why Cresta, and why contact-center AI specifically? [Likely]
    - What do you think is hardest about applying LLMs in a contact center compared with a consumer chatbot?
    - Which Cresta product would you most want to work on, and why?
2. Walk me through your background in two minutes, focusing on ML you've shipped to production. [Likely]
    - Which of these projects was most similar to real-time conversational AI?
3. What level are you targeting, and what scope did you own in your current role? [Likely]
    - How many engineers or models did your work touch?
4. What are you looking for in your next team: research-heavy, product-heavy, or platform? [Likely]
5. Are you comfortable working in a hybrid or in-office setup, and what is your timeline with other processes? [Likely]
6. What do you know about how Cresta's customers (enterprise contact centers) measure success? [Likely]
    - Which KPIs (AHT, CSAT, conversion, first-contact resolution) would you expect an ML feature to move?

B. Technical phone screen (coding)

7. Given a stream of transcript utterances with speaker labels and timestamps, merge consecutive utterances from the same speaker and output conversation turns. [Likely]
    - How do you handle overlapping timestamps or out-of-order events?

    - How would this change if events arrive in real time and you must emit partial results?
8. Implement a sliding-window function that returns the top-k most frequent phrases over the last N minutes of conversations. [Likely]
    - What is the memory complexity, and how would you bound it?
9. Implement a simple tokenizer plus n-gram counter, then use it to find the phrases that best distinguish converted from non-converted sales calls. [Likely]
    - How would you control for phrases that are just frequent?
10. Write a function that redacts PII (phone numbers, emails, card numbers) from transcripts. [Likely]
    - How would you test it? What are its precision/recall tradeoffs?
    - When would you replace regex with a learned NER model?
11. Implement beam search (or top-k / top-p sampling) over a given next-token probability function. [Likely]
    - How do you handle length normalization?
12. Implement precision, recall, F1 and a confusion matrix from scratch for multi-label intent classification. [Likely]
    - Micro vs macro averaging: which would you report to a customer?
13. Implement an LRU or TTL cache for embedding lookups. [Likely]
    - How does this change under concurrent access?
14. Given a list of call events with start/end times, compute the maximum number of concurrent calls (interval problem). [Likely]
15. Parse a nested JSON conversation log and compute per-agent metrics (average handle time, talk ratio, interruptions). [Likely]
    - How would you make this run over a billion records?
16. Implement cosine-similarity nearest-neighbor retrieval over a matrix of embeddings with NumPy, without loops. [Likely]
    - How would you scale this past memory? Which ANN index would you choose?

C. Onsite coding (production-quality)

17. Design and implement a rate limiter for calls to an LLM API shared across tenants. [Likely]
    - How do you enforce per-tenant fairness?
    - How do you handle retries and backoff?
18. Implement a streaming text-chunker that splits a live transcript into segments for downstream LLM calls under a token budget. [Likely]
    - How do you avoid splitting mid-sentence?
19. Implement a simple trie-based autocomplete for suggested agent responses ranked by frequency. [Likely]
    - How would you blend this with a model score?
20. Write a class that tracks a conversation state machine (greeting, verification, resolution, close) from a stream of classified utterances. [Likely]
    - How do you handle classifier noise that causes state flapping?
21. Implement a batching queue that groups incoming inference requests to maximize GPU throughput under a max-latency constraint. [Likely]
    - What is the tradeoff between batch size and p99 latency?

22. Implement edit distance and word error rate (WER) between a reference and a hypothesis transcript. [Likely]
    - How does WER fail to reflect downstream task quality?
23. Implement a deduplication routine for near-duplicate chat transcripts (e.g., MinHash or shingling). [Likely]
24. Refactor a given messy training-data pipeline function for testability. [Likely]
    - What unit tests would you write first?

D. ML system design (core onsite round)

25. Design a real-time agent-assist system that listens to a live call and suggests the next best response to the agent. [Likely]
    - What is your end-to-end latency budget from audio to suggestion?
    - Where do ASR, retrieval and LLM generation sit, and which of them streams?
    - How do you evaluate suggestion quality offline and online?
    - How do you decide when to show a suggestion versus stay silent?
26. Design a system that automatically scores 100% of calls against a QA rubric (automated QA). [Likely]
    - How do you turn a free-text rubric item into a reliable model judgment?
    - How do you calibrate against human QA scores, and handle disagreement between human graders?
    - How do you keep cost per call bounded?
27. Design an LLM-powered virtual agent that fully handles customer chats or voice calls for a specific enterprise. [Likely]
    - How do you ground it in the customer's knowledge base and policies?
    - How do you design escalation to a human?
    - What guardrails stop it from making unauthorized promises (refunds, pricing)?
    - How do you evaluate it before launch?
28. Design a system that discovers the behaviors of top-performing agents and turns them into coaching guidance. [Likely]
    - How do you separate causation from correlation in "what top performers do"?
29. Design a knowledge-assist (RAG) system that retrieves answers from a customer's documents during a live conversation. [Likely]
    - How do you chunk and index heterogeneous documents (PDFs, wikis, macros)?
    - How do you handle stale or conflicting documents?
    - What retrieval metrics and answer-faithfulness metrics would you track?
30. Design a topic/intent discovery pipeline over millions of unlabeled conversations for a new customer. [Likely]
    - How do you let non-technical customer admins name and refine the clusters?
31. Design a call-summarization feature that writes post-call notes into a CRM. [Likely]
    - How do you measure and reduce hallucinated facts?
    - How do you enforce a customer-specific summary template?
32. Design a multi-tenant model customization strategy: one base model with per-customer adaptation. [Likely]
    - When do you use prompting, retrieval, LoRA adapters or full fine-tuning?
    - How do you serve hundreds of adapters efficiently?
    - How do you prevent data leakage across tenants?

33. Design monitoring for production LLM features across many enterprise customers. [Likely]
    - What signals detect quality regressions when there are no labels?
    - How do you catch a regression that affects only one customer?
34. Design the data flywheel: how agent interactions with suggestions (accept, edit, ignore) feed back into model improvement. [Likely]
    - What biases does this implicit feedback introduce?
35. Design a real-time sentiment / escalation-risk detector for live calls. [Likely]
    - How do you combine acoustic and textual signals?
    - How do you set thresholds per customer?
36. Design the migration of a feature from an expensive frontier LLM API to a smaller self-hosted model. [Likely]
    - How do you decide the quality bar is met?
    - Distillation or fine-tuning: which, and on what data?
37. Design an evaluation platform the whole ML org uses to compare model versions before release. [Likely]
    - How do you build and maintain golden sets per customer and use case?
    - When can you trust LLM-as-judge, and how do you validate the judge?

E. ML depth: LLMs, NLP and speech

38. Walk through how you would fine-tune an LLM for a domain-specific classification or generation task with limited labeled data. [Likely]
    - SFT vs preference optimization (DPO/RLHF): when is each worth it?
39. How do you reduce hallucinations in a RAG system? [Likely]
    - How would you measure groundedness automatically?
40. Explain the main contributors to LLM inference latency and how to reduce them. [Likely]
    - KV cache, batching, quantization, speculative decoding: what does each trade off?
41. How do you handle very long conversations that exceed the context window? [Likely]
    - Summarization memory vs retrieval: what are the failure modes of each?
42. How does streaming ASR differ from offline ASR, and how do ASR errors propagate into downstream NLP? [Likely]
    - How would you make a downstream classifier robust to ASR errors?
43. How would you do speaker diarization or agent/customer channel separation, and how does it fail? [Likely]
44. How do you build an intent classifier when labels are noisy and categories change per customer? [Likely]
    - What are the tradeoffs among weak supervision, active learning and LLM-generated labels?
45. Compare embedding models for semantic search over conversational text. How would you fine-tune one? [Likely]
    - Which hard-negative mining strategy would you use?
46. How do you design prompts that are robust across customers and model versions? [Likely]
    - How do you version and regression-test prompts?

47. Explain calibration. Why does it matter when you show a confidence-gated suggestion to an agent? [Likely]
48. How do you detect and handle distribution shift when onboarding a new enterprise customer in a new vertical? [Likely]
49. How would you use LLMs to generate synthetic training data, and how do you keep it from degrading the model? [Likely]
50. How do you evaluate a generative model when there is no single correct answer? [Likely]
    - How do you design a rubric for human raters? How do you measure inter-annotator agreement?
51. Explain how you would structure tool use / function calling for an AI agent that needs to look up orders or reset passwords. [Likely]
    - How do you test multi-turn agent trajectories?
52. What are the risks of prompt injection in a customer-facing voice or chat agent, and how do you mitigate them? [Likely]
53. Explain transformer attention and its complexity. What changes make it work in long-context or streaming settings? [Likely]
54. How would you A/B test a new agent-assist model when the unit of randomization could be agent, call or customer? [Likely]
    - How do you handle network effects or learning effects among agents?
    - What sample size and duration would you need?

F. Project deep dive (senior bar)

55. Pick the most impactful ML system you've built. Walk me through the problem, your role, the architecture and the result. [Likely]
    - What was the baseline, and how did you prove the improvement?
    - What would you do differently now?
    - What part did you personally own, as opposed to the team?
56. Tell me about a model that worked offline but failed online. [Likely]
    - How did you diagnose the gap?
57. Describe a time you chose a simpler model over a more sophisticated one. [Likely]
58. How did you set up evaluation for your project, and what did your metrics miss? [Likely]
59. What were the biggest data-quality problems in your project, and how did you find them? [Likely]
60. How did you handle latency, cost or scaling constraints in production? [Likely]
    - What did serving cost per request look like, and how did you reduce it?
61. Describe how you deployed and monitored your model. What alerts existed, and did any fire? [Likely]
62. Have you shipped an LLM-based feature to production? What broke first? [Likely]
63. If we gave you this same problem in the contact-center domain, what would change? [Likely]
64. Describe a technical decision you made that you later reversed. [Likely]

G. Behavioral, values and collaboration

65. Tell me about a time you worked directly with a customer, or customer-facing teams, to define what an ML feature should do. [Likely]
    - How did you translate vague customer asks into measurable requirements?
66. Tell me about a time you owned a problem end to end without being asked. [Likely]
67. Describe a disagreement with a product manager or another engineer about model quality versus shipping speed. [Likely]
    - How was it resolved, and what did you concede?
68. Tell me about a time you had to deliver under heavy ambiguity at a fast-moving company. [Likely]
69. Tell me about a project that failed. What did you learn? [Likely]
70. Describe how you mentored a more junior engineer or raised the technical bar of your team. [Likely]
71. Tell me about a time you pushed back on a customer request or an executive priority because of data or technical risk. [Likely]
72. How do you prioritize between customer escalations, roadmap work and tech debt? [Likely]
73. Tell me about a time you influenced a decision across teams without direct authority. [Likely]
74. Describe a time you handled sensitive data responsibly (PII, compliance, enterprise security reviews). [Likely]
75. How do you stay current with fast-moving LLM research, and how do you decide what is worth adopting? [Likely]
76. Tell me about the hardest production incident you were involved in. [Likely]
    - What did the postmortem change?
77. What kind of feedback have you received that changed how you work? [Likely]
78. Where do you want to be in 3 years, and how does this role get you there? [Likely]

H. Hiring-manager and senior-scope probes

79. If you joined and found that our evaluation process for a key model was weak, what would you do in your first 90 days? [Likely]
80. How would you decide whether to build a capability in-house or use a third-party model provider? [Likely]
81. How do you scope an ML project so that it delivers value in weeks rather than quarters? [Likely]
82. How would you explain a model regression to an enterprise customer? [Likely]
83. What does good ML engineering hygiene look like (reproducibility, experiment tracking, data versioning)? [Likely]
    - Which of these have you introduced on a team yourself?
84. How do you balance research exploration with product delivery? [Likely]
85. How do you write a design doc for an ML system, and who reviews it? [Likely]
86. What would make you successful here, and what would make you fail? [Likely]

I. Domain sense: contact-center AI

87. What are the main sources of data in a contact center, and what quality problems does each have? [Likely]

88. How would you measure whether real-time guidance actually improves agent outcomes, rather than just correlating with good agents? [Likely]
89. How would you handle multilingual and code-switched calls? [Likely]
90. Which parts of a voice AI agent are most latency-sensitive (barge-in, turn-taking, TTS), and how would you design for them? [Likely]
    - How do you detect end of turn?
91. What compliance constraints (call recording consent, PCI, HIPAA, data residency) affect how you train and serve models? [Likely]
92. How do you design an ML product so that non-technical customer admins can configure behaviors (custom rules, topics, scorecards) safely? [Likely]
93. How would you prevent an automated QA score from being gamed by agents once they know the criteria? [Likely]

J. Questions you should ask them (prep)

94. Which team is this role on, and what are its next two quarters of goals? [Likely]
95. How does the team evaluate LLM features before they reach customers? [Likely]
96. What is the split between self-hosted models and third-party APIs, and where is it heading? [Likely]
97. What does the senior-to-staff promotion path look like for MLEs? [Likely]
98. How much direct customer contact do MLEs have? [Likely]

References

No sources could be retrieved in this run: web search was exhausted and the proxy blocked the target domains, so none are listed. No URLs or candidate reports have been invented. Recommended sources to check manually before the onsite (fill in URLs once verified):

1. Cresta official careers page and job postings for Senior / Staff Machine Learning Engineer (cresta.com). Not accessed.
2. Glassdoor: Cresta interview reviews, filtered by Machine Learning Engineer / Software Engineer. Not accessed.
3. Blind (teamblind.com): Cresta company page and interview threads. Not accessed.
4. Cresta engineering / AI blog posts on agent assist, AI agents and LLM evaluation. Not accessed.
5. LeetCode Discuss / 1point3acres: posts mentioning Cresta onsites. Not accessed.